Monitoring with top Extra

Skills you will learn

Version Control & Linux: Linux Fundamentals

1. Why learn `top` ?

`top` is the Swiss-army knife that ships with almost every Linux distribution. It lets you watch — in real time — how CPU, memory, and individual processes behave, so you can *pinpoint* bottlenecks before they become outages, *verify* that tuning changes had the intended effect, and *hunt down* runaway tasks without installing extra tools.

2. Launching & quick tour

```
top              # live, interactive view
top -d 2         # refresh every 2 s
top -p 1234,5678 # watch only PIDs 1234 and 5678
sudo top -H      # show kernel threads (-H) and root-only info
```

Copy Explain

Header (first 5 lines)

```
12:14:03  up 5 days,  3 users,  load average: 1.23, 0.77, 0.41      ← system time
Tasks: 218 total, 1 running, 216 sleeping, 1 stopped, 0 zombie       ← process st
%Cpu(s):  3.5 us, 1.2 sy, 0.0 ni, 94.7 id, 0.5 wa, 0.0 hi, 0.1 si   ← CPU breakd
MiB Mem :  15728 total,  2192 free,  9354 used,  4182 buff/cache     ← RAM statu
MiB Swap:   2048 total,  2031 free,    17 used.  5317 avail Mem      ← swap & “a
```

Process list (rest of the screen) – one line per PID, default sort is *CPU %*.

3. Key metrics to watch

Ask AI Tutor

Area	What to Monitor	Why it matters	Red-flag values
Load average	Three numbers = 1 min, 5 min, 15 min	> #CPU cores for extended periods = saturation	
%us / %sy	High *user* → app code; high *system* → kernel / I/O	Sudden jumps hint at bugs or I/O wait storms	
%wa	I/O wait time	> 20 % means disks or NFS are bottlenecked	
Free / available mem	Real RAM left after cache	If *available* → 0 and swap climbs, expect thrashing	
Swap-in / out columns (press *f* → select)*	Swap churn rate	Continuous swap-out + CPU idle ≈ memory pressure	
Process RES / %MEM	Resident set size of each task	Growing steadily → memory leak	

*Swap-in/out columns are hidden by default; hit **f**, then **p** and **o** to toggle them.

4. Essential interactive shortcuts

(press inside `top`; *case-sensitive*)

Key	Action	Notes
h	Help summary	Good first rescue if you forget
1	Per-CPU core usage	Vital on multi-core boxes
M / **P** / **T**	Sort by *memory*, *CPU*, or *time*	Cycle quickly to see hogs
u *username*	Filter by user	Debug a single student's VM
p *PID*	Filter by PID	Zoom in on one service
k *PID*	Kill prompt	Sends SIGTERM by default; use 9 for SIGKILL
r *PID nice*	Renice on the fly	Lower nice → higher priority
z	Toggle colour	Easier visual scanning
b	Bold high CPU tasks	Another readability tweak

Key	Action	Notes
I	Toggle Irix mode (per-CPU %)	Needed to interpret multicore correctly
W	Write your current layout to `~/.toprc`	Keeps custom columns & sort order

5. Real-world scenarios & playbooks

Situation	Quick checklist
CPU spike at 99 % (students running infinite loops)	1 → per-core view → is it one core? Press **P** to sort.Note PID; **k** to SIGSTOP (19) first, check effect, then SIGTERM (15) or SIGKILL (9).Renice competing services (`r pid 10`) if latency sensitive.
Memory leak suspicion	Press **M**. Look for a PID whose *RES* climbs every refresh. Toggle **f** → add `SWAP` column; if swap rises in parallel, you've confirmed the leak.
I/O bottleneck	In header, `%wa` climbs; CPU idle high but tasks slow. Add `IO` columns: **f** → enable `IO_READ` / `IO_WRITE` . Offending process often shows near-constant writes/reads. Time to check disks with `iotop` / `iostat` .
Zombie cleanup	Header shows non-zero *zombie* count. Press **Z** (uppercase) to highlight zombie lines, note PPID, restart parent service cleanly.
Kernel causing trouble after update	`%sy` high while `%us` low. Verify with `perf top` or `strace -p pid` . Roll back or patch kernel.

6. Customizing & persisting layouts

1. Press **f** → toggle columns (letters shown on screen).
2. Press **o**/**O** to move a field *left/right* to your preferred position.
3. Sort field permanently: e.g. press **F** then **m** for memory.
4. Press **W** to save as default for your user.
5. Copy `~/.toprc` into student accounts for a common classroom view.

7. Batch & logging modes

```
top -b -n 60 -d 1 > /tmp/top.log        # capture 1 min (60 iterations)
grep --line-buffered firefox /tmp/top.log | awk '{print $9}' | ...
```

Copy Explain

Why? Automate baseline collection before and after a deployment, or feed numbers into Grafana/Loki later.

8. Production safety tips

- Run `top` inside **`tmux`** / **`screen`** when SSHing, so the session survives drops.

- Always try **SIGSTOP (19)** before SIGKILL in multi-tenant environments — killing mid-write can corrupt data.
- Teach juniors to read *load average* **in context of CPU count** (e.g. load = 4 on a 32-core host is fine).
- Save your layout once, sync `~/.toprc` via dot-files repo. Consistency speeds incident response.
- Use `htop` for a friendlier view, but default to `top` on minimal rescue environments.

9. Recap & next steps

You now know how to:

1. Launch `top` in various modes and interpret its header.
2. Spot CPU, memory, and I/O red flags quickly.
3. Use the most common interactive keys to filter, sort, renice, and kill tasks.
4. Apply real-life troubleshooting recipes in the field.
5. Persist a custom layout and export data for later analysis.

Practice drill: Spin up a small VM, start a `stress --cpu 4 --vm 2` workload, and time how fast you can (a) detect the spike, (b) identify the PID, (c) reduce its priority, and (d) verify the system stabilizes. Aim for under **90 seconds**.

Happy monitoring — may your load stay low and your zombies rest in peace!

AI TUTOR ACTIONS

Generate a practice quiz Summarize this unit Ask about this unit

Complete unit